Software Architecture and Design Specification (SAD)

Change Management Software

Project	Change Management Software (CMS)
Version	2.0 (as built)
Authors	Team 4: Vivaan Atharva (PES1UG24AM071), Dhanush S S (PES1UG24AM082), Gayathri Priya V (PES1UG24AM105), Gonu Guntla Hema Kowshik (PES1UG24AM106)
Date	7 November 2026
Status	Final: describes the release built in Sprints 1 and 2 (feature freeze 7 Nov 2026)

Revision History

Version	Date	Author	Change summary	Approval
1.0	28 October 2026	Team 4	Initial project-specific draft	Team
2.0	7 November 2026	Team 4	Updated to the implemented system: as-built technology, design and test results; implementation status and deviations recorded	Team

Approvals

Role	Name	Signature / Email	Date
Team lead	Vivaan Atharva		
Course instructor / TA	To be filled by instructor	Pending	

Table of Contents

- 1. Introduction
- 2. Architecture Goals and Constraints
- 3. Stakeholders and Concerns
- 4. Architecture and Components
- 5. Architecture Pattern and Technology Stack
- 6. Data Design
- 7. Routes and Service Design
- 8. Sequence Flows
- 9. Security Architecture
- 10. Error Handling, Logging and Monitoring
- 11. Traceability, Risks and Open Issues

1. Introduction

1.1 Purpose

This document describes the architecture and design of CMS as built at the end of Sprint 2. It defines component responsibilities, data relationships, the change request workflow model, service boundaries, important flows, security controls, and traceability to the SRS. Version 1.0 was the proposed design; version 2.0 records the implemented modules, routes and tables, and marks the parts of the original design that were not built (see 11.2).

1.2 Scope

The design covers change request creation and editing, the status workflow, review and approval, assignment, comments and work notes, resolution, verification and closure, reopening, SLA/overdue tracking, in-app notifications, reports, authentication, authorization, and audit history. Email, chat, source-control and CI/CD integrations are optional future work.

2. Architecture Goals and Constraints

- Correct, consistently enforced workflow transitions and a complete, traceable history are primary goals.
- Keep workflow and assignment rules in one place and testable independently of UI code.
- Use a small, understandable architecture suitable for a four-member student team and sprint delivery.
- Persist data reliably, validate all user input, and prevent concurrent updates from silently overwriting each other.
- Match final diagrams and technology descriptions to the implementation; record deviations before submission.

3. Stakeholders and Concerns

Stakeholder	Primary concern
Requester	Quick CR creation, visibility of progress, ability to confirm or reopen.
Change manager / reviewer	Clear triage queue, fair workload assignment, overdue visibility, reports.
Engineer	Clear list of assigned work, easy status updates, full context on each CR.
Administrator	User access, configuration of components and SLA targets, auditability.
Developers	Modular code, testable workflow rules, clear interfaces.
Evaluator / TA	Traceability from requirements to implementation, tests, commits, and demos.

4. Architecture and Components

4.1 Chosen Pattern

Use a layered modular-monolith architecture: presentation/UI, application routes/controllers, business services, repositories/data-access, and relational database. The workflow rules are implemented as an explicit state machine inside the business layer so every status change goes through one tested component. This keeps deployment and collaboration simple while allowing service-level unit tests. A microservices architecture is not recommended for the initial prototype because it would add operational overhead without a demonstrated need.

4.2 Component Responsibilities (as built)

Component	Module(s)	Responsibilities	Status
Presentation layer	cms/templates/ (Jinja + Bootstrap 5.3)	Dashboard, CR list and detail pages, create/edit form, action buttons per role, status, assignment and edit history.	Built
Routes (controllers)	cms/routes/main, auth, change_requests, assignment, workflow	Map URLs to services; check login and role; flash user-facing messages; render pages.	Built
Authentication / authorization	cms/services/auth_service.py, cms/decorators.py, cms/security.py	Sign-in/out with Flask-Login, roles_required decorator (Admin passes every check), CSRF token on every form, safe redirect check.	Built
Change request service	cms/services/ change_request_service.py	Validate, create, search/filter, edit (with edit history), record-level visibility, due date from priority.	Built
Workflow engine	cms/services/ workflow_service.py	Single TRANSITIONS table; checks role, person and required input; writes status and history in one commit; allowed_next() drives the buttons.	Built
Assignment service	cms/services/ assignment_service.py	Assign/reassign to active engineers only; writes AssignmentHistory and calls the workflow engine.	Built
SLA logic	change_request_service.due_date_for / set_priority; ChangeRequest.is_overdue	Due date from SLA days per priority; overdue = past due and not Resolved/Closed/Rejected.	Built
Dashboard service	cms/services/ dashboard_service.py	Counts by status and priority, work lists per role, overdue list.	Built
Comment / activity service	-	Comments and work notes.	Not built (CMS-10)
Notification service	-	In-app notifications.	Not built (CMS-14)
Reporting service	-	Open-CR, ageing and resolution-time reports.	Not built
Audit service	History tables	Status, assignment and edit history act as the audit trail for CRs.	Partial
Data access	Flask-SQLAlchemy models in cms/models/	Tables, relationships, constraints.	Built
Database	SQLite file instance/cms.db (in-memory for tests)	Persist users, CRs and history.	Built

4.3 Component Diagram (text specification)

Browser → Flask routes (main, auth, change_requests, assignment, workflow) → services (auth, change request, workflow, assignment, dashboard) → Flask-SQLAlchemy models → SQLite. Routes never change a status directly: the workflow routes and the assignment service both call workflow_service.transition(). The dashboard service only reads data. The component diagram in docs/diagrams/component.png shows the same structure.

5. Technology Stack and Data Stores

Part	Choice (as built)
Language	Python 3.12 (3.10 or newer supported)
Web framework	Flask 3 with server-rendered Jinja templates; Bootstrap 5.3 for layout

Database	SQLite through Flask-SQLAlchemy 3.1; tables created on start-up
Sessions	Flask-Login; passwords hashed with Werkzeug (salted)
CSRF protection	Project module cms/security.py: a per-session token checked on every POST
Command line	flask --app cms init-db | reset-db | seed (demo users and 10 demo change requests)
Tests	pytest; 337 automated tests
CI	GitHub Actions on Ubuntu, Windows and macOS with Python 3.12 for every pull request and push to main
Tracking	Jira Scrum project CMS (two sprints); GitHub feature branches and pull requests

The design proposed a JSON API and PostgreSQL/MySQL for shared use. The team chose server-rendered pages and SQLite to keep setup to a single pip install on every member's laptop; the layered design means a different database can be configured through SQLALCHEMY_DATABASE_URI without code changes.

6. Data Design

Table (model)	Important fields	Relationships / rules
users (User)	id, name, email (unique), password_hash, role, active, created_at	Role is one of Requester, Reviewer, Engineer, Admin. Inactive users cannot sign in or be assigned.
change_requests (ChangeRequest)	id, cr_number (unique, CR-0001), title, description, cr_type, priority, component, status, requester_id, assignee_id, due_date, resolution_summary, created_at, updated_at, closed_at	One requester, at most one assignee. Status changes only through the workflow engine.
status_history (StatusHistory)	id, change_request_id, from_status, to_status, reason, user_id, created_at	Append-only; one row per status change, including the initial Submitted row.
assignment_history (AssignmentHistory)	id, change_request_id, from_user_id, to_user_id, assigned_by_id, created_at	Append-only; one row per assignment or reassignment.
edit_history (EditHistory)	id, change_request_id, field, old_value, new_value, user_id, created_at	Append-only; one row per field changed through the edit page.

Type, priority, component and status values are fixed lists in cms/constants.py and are checked by the services. SLA days per priority come from the SLA_DAYS setting (default Critical 1, High 3, Medium 5, Low 10; override with the CMS_SLA_DAYS environment variable). The Comment, Notification, SlaPolicy and AuditEvent tables in the original design were not built.

6.1 Workflow State Model

From status	Allowed next status	Who / required input
Submitted	Under Review	Reviewer
Under Review	Approved or Rejected	Reviewer; rejection reason required
Approved	Assigned	Reviewer; active engineer required
Assigned	In Progress; or reassigned (stays Assigned)	Assigned engineer starts work; reviewer reassigns
In Progress	Resolved; or Assigned on reassignment	Assigned engineer with a resolution summary; reviewer reassigns
Resolved	Closed or Reopened	Requester or reviewer; reopen reason required
Reopened	In Progress	Assigned engineer
Rejected / Closed	None (terminal)	No further changes; editing is also blocked

6.2 Data Integrity Rules

- Use database constraints (foreign keys, NOT NULL, allowed values) where supported and repeat important validation in the service layer.
- Change a CR's status only through the Workflow Engine, and write the status change and its history row in one database transaction.
- Optimistic locking with a version column was designed but not built (SRS CMS-NF-007); the last save wins.

- Keep StatusHistory, AssignmentHistory and AuditEvent append-only.
- Timestamps are stored in UTC; due dates are calendar dates compared with the server's current date.

7. Routes and Service Design (as built)

CMS uses server-rendered HTML pages and form posts rather than a JSON API. Every POST carries a CSRF token. Errors are shown as flash messages or friendly 400/403/404 pages.

Operation	Route	Who	Result / errors
Sign in / out	GET, POST /login; POST /logout	Anyone / signed-in user	Session created; same message for any bad login; inactive users refused.
Dashboard	GET /	Signed-in user	Counts, work lists for the role, overdue list.
List / search CRs	GET /change-requests	Signed-in user	Filters q, status, priority, component, assignee, requester, overdue; sort. Requesters see only their own CRs.
Create CR	GET, POST /change-requests/new	Signed-in user	Redirect to the new CR; 400 with field errors.
CR detail	GET /change-requests/<id>	Signed-in user with access	Fields, buttons allowed for this user, history; 403 / 404.
Edit CR	GET, POST /change-requests/<id>/edit	Requester or reviewer	Saved with edit history; 400 invalid; 403 if Closed/Rejected or not permitted.
Review	POST .../<id>/start-review, /approve, /reject (reason)	Reviewer	Status updated; reject needs a reason; 403 wrong role.
Assign / reassign	POST .../<id>/assign (assignee_id)	Reviewer	Assigned to an active engineer; 403 for other roles.
Work	POST .../<id>/start-work, /resolve (resolution_summary)	Assigned engineer	Status updated; resolve needs a summary.
Closure	POST .../<id>/close, /reopen (reason)	Requester or reviewer	Closed (sets closed_at) or Reopened with a reason.
Health check	GET /health	Anyone	{"status": "ok"}

8. Sequence Flows

8.1 Create Change Request

- Requester opens the new-request form and submits title, description, type, priority and component.
- UI sends the request to the route/controller.
- Controller checks authentication, then calls Change Request Service.
- Service validates required fields and allowed values, generates the CR number, and sets status Submitted and due date from the SLA policy.
- Service writes the CR and its first StatusHistory row in one database transaction.
- The new CR appears in the reviewers' "Waiting for review" list on the dashboard; the user is redirected to it.
- If validation or persistence fails, the transaction is rolled back and a safe error is returned.

8.2 Approve, Assign and Resolve

- Reviewer opens a Submitted CR, clicks Start review, then Approve.
- Reviewer selects an engineer; Assignment Service checks the user is an active engineer and calls the Workflow Engine to move the CR to Assigned.
- The CR update, StatusHistory and AssignmentHistory rows are written in one transaction; the CR appears in the engineer's "Assigned to me" list.
- Engineer starts work (In Progress) and submits a resolution summary to move the CR to Resolved.
- Workflow Engine rejects the resolve step if the summary is empty or the caller is not the assignee.

8.3 Verify and Close or Reopen

- Requester or reviewer opens a Resolved CR and reviews the resolution summary.

- To close, they submit the Closed transition; the closure date is set and the CR leaves the open and overdue lists.
- To reopen, they submit the Reopened transition with a reason; the CR returns to the same engineer's list.
- A history row is written for either outcome and the dashboard counts reflect the change.

9. Security Architecture

Threat / concern	Control
Unauthorized access	Authentication plus server-side role checks for every protected operation.
Acting on someone else's CR	Record-level checks: only the assignee can resolve; only the requester or a reviewer can close or reopen.
Tampering with workflow or history	All status changes go through the Workflow Engine; history and audit tables are append-only.
Credential disclosure	Salted password hashing, HTTPS in deployed environments, secret-free logs.
Injection / cross-site scripting	Parameterized database access, strict input validation, and output escaping of user-entered text.
Privilege escalation	Deny-by-default permissions and tests for direct requests by unauthorized roles.
Cross-site request forgery	Per-session CSRF token checked on every POST (cms/security.py).
Lost updates	Not handled in this release (last save wins); optimistic locking is future work.
Accidental data loss	SQLite file in instance/; back up by copying the file. reset-db asks for confirmation.

Threat-modeling prompts: spoofing (fake user/session), tampering (changing status or history outside the workflow), repudiation (denial of an approval, rejection or closure), information disclosure (credentials or CR content exposed to the wrong user), denial of service (invalid or excessive requests), and elevation of privilege (engineer or requester performing reviewer/admin operations).

10. Error Handling, Logging and Monitoring

- Return consistent validation, permission, transition and conflict errors; do not show stack traces to end users.
- Log unexpected failures with a correlation/reference ID where feasible; never log passwords or secrets.
- Record business audit events separately from diagnostic logs.
- Monitor application startup failures, failed operations, rejected transitions, version conflicts, and test results.
- Provide an understandable empty state when no CRs, notifications or report rows match the selected filter.

11. Traceability, Risks and Open Issues

SRS requirement	Design component (as built)
CMS-F-001, F-002 / SR-001 to SR-003	auth routes, auth_service, roles_required, Flask-Login, security.py
CMS-F-003 to CMS-F-006	change_requests routes, change_request_service, EditHistory
CMS-F-007, F-008, F-011, F-012	workflow routes, workflow_service (TRANSITIONS), StatusHistory
CMS-F-009 / CMS-SR-005	assignment route, assignment_service, AssignmentHistory, record-level checks
CMS-F-010	workflow start-work route (comments not built)
CMS-F-013	StatusHistory, AssignmentHistory, EditHistory shown on the detail page
CMS-F-014	due_date_for, set_priority, ChangeRequest.is_overdue, overdue list filter
CMS-F-016	dashboard_service and dashboard page
CMS-F-015, F-017, F-018	Not built
CMS-NF-005	Layered routes / services / models; services tested without HTTP
CMS-NF-006	README setup steps; CI on three operating systems

11.1 Risks and Mitigations

Risk	Mitigation
Workflow rules scattered across UI and routes	Keep one transition table in the Workflow Engine; unit-test every allowed and disallowed transition.
History out of sync with current status	Write status and history in one transaction; add a consistency test.
Due-date and time-zone errors	Store UTC, define local-time rules, and test due today, due tomorrow, and past-due cases.
Merge conflicts across four contributors	Assign clear module ownership, use feature branches and pull requests, integrate frequently.
Scope expansion (email, Git integration, attachments)	Prioritize the core create/assign/resolve/close flow; label integrations as future work.
Requirements differ from delivered product	Maintain RTM and document deviations for final demonstration.

11.2 Decisions Taken and Remaining Gaps

- Framework and database: Flask with SQLite (decided in Sprint 1; see section 5).
- File attachments on CRs: not included in this release.
- Notifications: not built; the dashboard work lists show each user what needs their attention.
- Reassigning an In Progress CR: allowed for reviewers; the CR returns to Assigned for the new engineer.
- Not built from the original design: comments and work notes, notifications, reports, admin settings screens, optimistic locking, and separate audit events. These are listed as deviations in SRS section 10.
- Diagrams: docs/UML.md and docs/diagrams/ use the actual module names.